

MLOps Assignment 1: Multi-Container Application Deployment with CI/CD

1 Objective

In this assignment, you will develop a multi-container application consisting of a Flask web application and a MongoDB database. You will containerize these services using Docker, implement a CI/CD pipeline with GitHub Actions, and deploy them on Kubernetes using Minikube, with proper configuration management using ConfigMaps and Secrets.

2 Prerequisites

- Understanding of Python and Flask
- Knowledge of Docker containerization
- Understanding of Kubernetes concepts
- GitHub account
- Installed tools:
 - Docker
 - Minikube
 - kubectl
 - Git

3 Part 1: Application Development (15 points)

1. Create a simple Task Manager application using Flask with the following features:
 - View a list of tasks
 - Add new tasks
 - Mark tasks as complete
 - Delete tasks
2. Use MongoDB as the database:
 - Install and use PyMongo library
 - Configure connection using environment variables
 - Create functions to interact with the database
3. Create a basic UI using HTML/CSS (can use Bootstrap if desired)

4 Part 2: Containerization with Docker (25 points)

1. Create a Dockerfile for your Flask application:
 - Use an appropriate Python base image
 - Install dependencies from requirements.txt
 - Copy application code
 - Configure environment variables
 - Set the entry point
2. Create a docker-compose.yml file with two services:
 - Flask web application
 - MongoDB database
 - Configure networking between containers
 - Set up environment variables for MongoDB connection
3. Build and test your multi-container setup locally

5 Part 3: GitHub Actions CI/CD Pipeline (30 points)

1. Create and maintain a `dev` branch for development:
 - Develop and test new features directly on the `dev` branch incrementally.
 - Ensure each push triggers continuous integration checks.
2. Once you have completed a new feature or set of features:
 - Generate a Pull Request (PR) from `dev` branch to the `main` branch.
 - Merge this PR after reviewing and ensuring CI checks pass.
3. Create a GitHub Actions workflow file (`.github/workflows/ci-cd.yml`) that:
 - Triggers Continuous Integration on pushes to the `dev` branch:
 - Performs Python code linting.
 - Builds the Docker image to confirm successful builds.
 - Triggers Continuous Deployment upon merging PRs into the `main` branch:
 - Performs linting again for validation.
 - Builds and tags the Docker image appropriately.
 - Logs into Docker Hub using provided secrets.
 - Pushes the Docker image to Docker Hub.
 - Updates Kubernetes deployment manifest files with the new Docker image tag automatically.
 - Uses an automated script to update the Kubernetes cluster with the new deployment.
4. Set up GitHub Secrets in your repository settings:
 - `DOCKER_HUB_USERNAME`
 - `DOCKER_HUB_ACCESS_TOKEN`
 - `FLASK_SECRET_KEY`
 - `MONGO_INITDB_ROOT_USERNAME`
 - `MONGO_INITDB_ROOT_PASSWORD`

6 Part 4: Kubernetes Deployment (30 points)

1. Create the following Kubernetes manifest files:
 - `flask-deployment.yaml`: Define a deployment for your Flask application
 - `mongodb-deployment.yaml`: Define a deployment for MongoDB
 - `flask-service.yaml`: Expose your Flask application on a NodePort
 - `mongodb-service.yaml`: Create a service for MongoDB (ClusterIP)
 - `configmap.yaml`: Store non-sensitive configuration
 - `secret.yaml`: Store sensitive information securely
2. In your ConfigMap, include:
 - MongoDB connection information (host, port, database name)
 - Debug mode setting
 - Other application configurations
3. In your Secret, securely store:
 - Flask Secret Key
 - MongoDB username and password
4. Deploy your application on Minikube:
 - Start Minikube
 - Apply your manifest files
 - Verify both containers are running
 - Access the application through the Flask service

7 Submission Requirements

1. GitHub repository containing:
 - Complete application code with UI
 - Dockerfile and `docker-compose.yml`
 - GitHub Actions workflow file
 - Kubernetes manifest files
 - README.md with setup and deployment instructions
2. A brief report (maximum 2 pages) that includes:
 - Screenshot of the working application UI
 - Screenshot of successful GitHub Actions workflow run
 - Commands used for Kubernetes deployment
 - Explanation of how the two containers communicate
 - Explanation of how ConfigMaps and Secrets are utilized
 - Any challenges encountered and how you resolved them

8 Grading Criteria

- **Application (15%)**: Multi-container application works as expected
- **Docker Setup (25%)**: Both containers are properly configured and communicate
- **GitHub Actions (30%)**: CI/CD pipeline functions correctly with secrets
- **Kubernetes Deployment (30%)**: Application runs on Kubernetes using ConfigMaps and Secrets

9 Helpful Implementation Tips

1. MongoDB Connection String Format:

```
mongodb://username:password@hostname:port/database
```

2. Using environment variables in Flask:

```
import os
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

# Get MongoDB connection string from environment variable
mongo_uri = os.environ.get('MONGO_URI', 'mongodb://localhost:3000/DB')
client = MongoClient(mongo_uri)
db = client.get_default_database()
```

3. Creating Kubernetes secrets:

```
# Use base64 encoding for secrets
echo -n 'my-secret-password' | base64
```

4. Accessing ConfigMap values in deployment:

```
env:
  - name: MONGO_HOST
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: mongo-host
```

5. Accessing Secret values in deployment:

```
env:
  - name: MONGO_PASSWORD
    valueFrom:
      secretKeyRef:
        name: app-secrets
        key: mongo-password
```

Additional Helpful Hints for Kubernetes Deployment

1. Automating Kubernetes Updates via Script or GitHub Actions:

- You can automate the Kubernetes update process after pushing new Docker images by using the following commands:

```
kubectl apply -f ./k8s-manifests/  
kubectl rollout restart deployment flask-deployment
```

- Incorporate this script into your GitHub Actions workflow to automate the deployment process whenever code is merged into the `main` branch.

2. Using GitHub Actions for Automated Deployment:

- Consider using the `kubectl` action in GitHub Actions to automate your Kubernetes updates directly:

```
- name: Deploy to Kubernetes  
  uses: actions-hub/kubectl@master  
  env:  
    KUBECONFIG: ${ secrets.KUBECONFIG }  
  with:  
    args: apply -f ./k8s-manifests/
```

- Here, `KUBECONFIG` is stored as a secret in your GitHub repository, containing the configuration needed for GitHub Actions to securely access your Kubernetes cluster.

3. Ensuring Zero Downtime with Kubernetes Rollouts:

- Kubernetes supports zero-downtime deployments using a **RollingUpdate** strategy. This means new pods are launched and become healthy before the old pods are terminated, minimizing downtime.
- Default rollout strategy example:

```
spec:  
  strategy:  
    type: RollingUpdate  
    rollingUpdate:  
      maxSurge: 25%  
      maxUnavailable: 25%
```

- With these settings, Kubernetes ensures at most 25% of pods are unavailable during updates, allowing your service to remain continuously accessible.

4. Image Tags Best Practice:

- Avoid using the `latest` tag for production deployments. Instead, use unique tags such as commit hashes, build numbers, or semantic versioning to precisely control rollouts and rollbacks.
- Example tagging in GitHub Actions workflow:

```
docker build -t yourusername/flask-app:${ github.sha } .  
docker push yourusername/flask-app:${ github.sha }
```

- Update Kubernetes deployment manifests dynamically with these tags to keep track of deployments effectively.

Good luck!